

MongoDB CRUD Operations Using Mongosh

1. Database & Collection Creation

Creation of Database and Collection with Initial Data Insertion

```
mongosh mongod
Microsoft Windows [Version 10.0.22631.6199]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Golden Roots>mongosh
Current Mongosh Log ID: 69f57c61e9f334123682d0
Connecting to:  mongosh://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using Mongosh:  2.8.2
Using MongoDB:  6.0.7
Mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

The server generated these startup warnings when booting
2026-05-01T17:03:45.971+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted

test> use studentDB
switched to db studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB>
```

2. Insert Operations

Adding Single and Multiple Documents into the Collection

```
{ ok: 1 }
studentDB> db.students.insertOne({
  name: "priya",
  age: 21,
  course: "MERN Stack",
  status: "ongoing"
})
{ acknowledged: true,
  insertedId: ObjectId('69f725ce4710a94399abc114') }
studentDB> db.students.insertMany([
```

```
]
{
  acknowledged: true,
  insertedId: ObjectId('69f725ce4710a94399abc114')
}
studentDB> db.students.insertMany([
  {name: "arun", age: 34, course: "MERN Stack", status: "ongoing"},
  {name: "vishal", age: 25, course: "JAVA", status: "completed"},
  {name: "nareu", age: 18, course: "Python", status: "completed"}
])
{ acknowledged: true,
  insertedIds: {
    0: ObjectId('69f726004710a94399abc115'),
    1: ObjectId('69f726004710a94399abc116'),
    2: ObjectId('69f726004710a94399abc117')
  }
}
studentDB> db.students.find()
{
  _id: ObjectId('69f725ce4710a94399abc114'),
  name: 'priya',
  age: 21,
  course: 'MERN Stack',
  status: 'ongoing'
},
{
  _id: ObjectId('69f726004710a94399abc115'),
  name: 'arun',
  age: 34,
  course: 'MERN Stack',
  status: 'ongoing'
},
{
  _id: ObjectId('69f726004710a94399abc116'),
  name: 'vishal',
  age: 25,
  course: 'JAVA',
  status: 'completed'
},
{
  _id: ObjectId('69f726004710a94399abc117'),
  name: 'nareu',
  age: 18,
  course: 'Python',
  status: 'completed'
}
studentDB> db.students.find({course: "MERN Stack"})
```

3. Read Operations

Retrieving and Filtering Data from the Collection

```
mongosh mongodb://127.0.0.1:27027/
> use studentDB
studentDB> db.students.find()
[
  {
    _id: ObjectId('69f725ce4718a94399abc114'),
    name: 'priya',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f726804718a94399abc115'),
    name: 'arun',
    age: 34,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f726804718a94399abc116'),
    name: 'valar',
    age: 25,
    course: 'JAVA',
    status: 'completed'
  },
  {
    _id: ObjectId('69f726804718a94399abc117'),
    name: 'narmu',
    age: 18,
    course: 'Python',
    status: 'completed'
  }
]
studentDB> db.students.find({course:"MERN Stack"})
[
  {
    _id: ObjectId('69f725ce4718a94399abc114'),
    name: 'priya',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f726804718a94399abc115'),
    name: 'arun',
    age: 34,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
```

4. Update Operations

Modifying Existing Documents in the Collection

```
studentDB> db.students.updateOne({name:"arun"},
| {$set:{status:"completed"}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.updateMany(
| { course: "MERN Stack" },
| { $set: { status: "inprogress" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

5. Delete Operations

Removing Documents Based on Conditions

```
studentDB> db.students.deleteOne({ name: "priya" })
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 3 }
studentDB> db.students.insertMany([
| {name:"arun",age:34,course:"MERN Stack",status:"ongoing"},
| {name:"valar",age:25,course:"JAVA",status:"completed"},
| {name:"narmu",age:18,course:"Python",status:"completed"}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f729234718a94399abc118'),
    '1': ObjectId('69f729234718a94399abc119'),
    '2': ObjectId('69f729234718a94399abc11a')
  }
}
```

6. Query Operators

Applying Conditional Queries for Data Filtering

```
studentDB> db.students.find({ course: { $in: ["Java", "Python"] } })
```

```
[
  {
    _id: ObjectId('69f729234710a94399abc11a'),
    name: 'narmu',
    age: 18,
    course: 'Python',
    status: 'completed'
  }
]
```

```
studentDB> db.students.find({
  $and: [
    { course: "MERN Stack" },
    { status: "ongoing" }
  ]
})
```

```
[
  {
    _id: ObjectId('69f729234710a94399abc118'),
    name: 'arun',
    age: 34,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
```

```
studentDB> db.students.find({
  $or: [
    { course: "Java" },
    { age: { $lt: 21 } }
  ]
})
```

```
[
  {
    _id: ObjectId('69f729234710a94399abc11a'),
    name: 'narmu',
    age: 18,
    course: 'Python',
    status: 'completed'
  }
]
```

```
studentDB> db.students.find({ age: { $exists: true } })
```

```
[
  {
    _id: ObjectId('69f729234710a94399abc118'),
    name: 'arun',
    age: 34,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f729234710a94399abc119'),
    name: 'valar',
    age: 25,
    course: 'JAVA',
    status: 'completed'
  },
  {
    _id: ObjectId('69f729234710a94399abc11a'),
    name: 'narmu',
    age: 18,
    course: 'Python',
    status: 'completed'
  }
]
```

7. Use Case: E-Commerce Real-World Application of MongoDB Operations

Real-World Application of MongoDB Operations

```
test> use library
switched to db library
library> db.createCollection("books")
{ ok: 1 }
library> db.books.insertMany([
  { title: "Java Basics", author: "John", available: true, price: 500 },
  { title: "Python Guide", author: "Sara", available: false, price: 700 },
  { title: "MongoDB Mastery", author: "Alex", available: true, price: 600 }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f72cedef1b0512d5abc114'),
    '1': ObjectId('69f72cedef1b0512d5abc115'),
    '2': ObjectId('69f72cedef1b0512d5abc116')
  }
}
library> db.books.find({ available: true })
[
  {
    _id: ObjectId('69f72cedef1b0512d5abc114'),
    title: 'Java Basics',
    author: 'John',
    available: true,
    price: 500
  },
  {
    _id: ObjectId('69f72cedef1b0512d5abc116'),
    title: 'MongoDB Mastery',
    author: 'Alex',
    available: true,
    price: 600
  }
]
```

```
library> db.books.updateOne(
  { title: "Python Guide" },
  { $set: { available: true } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
library> db.books.deleteOne({ title: "Java Basics" })
{ acknowledged: true, deletedCount: 1 }
```

```
library> db.books.find()
[
  {
    _id: ObjectId('69f72cedef1b0512d5abc115'),
    title: 'Python Guide',
    author: 'Sara',
    available: true,
    price: 700
  },
  {
    _id: ObjectId('69f72cedef1b0512d5abc116'),
    title: 'MongoDB Mastery',
    author: 'Alex',
    available: true,
    price: 600
  }
]
library> |
```

Conclusion:

MongoDB CRUD operations were successfully implemented using mongosh. This exercise helped in understanding how to create, manage, and manipulate data efficiently in a NoSQL database environment.